

COMPTE RENDU TP2.3 (GESTION DES VULNÉRABILITÉS) : Audit de sécurité avec Nessus et OpenVAS

I- INTRODUCTION

La gestion des vulnérabilités est le pilier de la sécurité offensive et défensive. Elle consiste à identifier, classer et remédier aux faiblesses logicielles avant qu'elles ne soient exploitées par un attaquant. L'objectif de ce TP est de maîtriser les deux outils leaders du marché : **Nessus Essentials** et **OpenVAS**. Nous allons comparer leur efficacité sur un parc hybride (Linux & Windows) et analyser les résultats pour proposer un plan de remédiation.

1- Présentation des outils d'audit

a- Nessus Essentials (Tenable)

Nessus est le scanner de vulnérabilités le plus utilisé au monde. Développé par Tenable, il est réputé pour sa précision (faible taux de faux positifs) et sa base de données de plugins constamment mise à jour.

- Points forts : Interface intuitive, rapports détaillés, intégration facile avec d'autres solutions SOC.
- Usage dans le TP : Identification des failles critiques sur les OS et services réseau.

b- OpenVAS / GVM (Greenbone)

OpenVAS est l'alternative open-source la plus puissante. Il fait partie de la suite Greenbone Vulnerability Management (GVM).

- Points forts : Totalement gratuit, hautement personnalisable, idéal pour les infrastructures sans budget de licence propriétaire.
- Usage dans le TP : Scan comparatif et analyse des scripts de détection (NVT).

2- Étape 1 : Préparation de l'environnement

Avant de scanner, nous devons configurer notre laboratoire pour assurer la communication entre le scanner et les cibles.

- Machines Cibles : **Ubuntu** (192.168.56.10) et **Windows** (192.168.56.20).
- Réseau : Configuration des VMs en mode "Host-Only" pour isoler le trafic de scan.
- Vérification : Test de ping entre le scanner (Kali Linux) et les cibles.

```
fotsing@vbox: ~  
Session Actions Éditer Vue Aide  
inet 192.168.56.30/24 brd 192.168.56.255 scope global noprefixroute eth0  
    valid_lft forever preferred_lft forever  
inet6 fe80::1d2:a763:b258:e883/64 scope link noprefixroute  
    valid_lft forever preferred_lft forever  
  
(fotsing@vbox)-[~]  
$ ping 192.168.56.20  
PING 192.168.56.20 (192.168.56.20) 56(84) bytes of data.  
From 192.168.56.30 icmp_seq=1 Destination Host Unreachable  
^C  
— 192.168.56.20 ping statistics —  
6 packets transmitted, 0 received, 100% packet loss, time 5094ms  
pipe 4  
  
(fotsing@vbox)-[~]  
$ ping 192.168.56.20  
PING 192.168.56.20 (192.168.56.20) 56(84) bytes of data.  
64 bytes from 192.168.56.20: icmp_seq=1 ttl=128 time=3.39 ms  
64 bytes from 192.168.56.20: icmp_seq=2 ttl=128 time=1.63 ms  
64 bytes from 192.168.56.20: icmp_seq=3 ttl=128 time=1.79 ms  
64 bytes from 192.168.56.20: icmp_seq=4 ttl=128 time=0.927 ms  
^C  
— 192.168.56.20 ping statistics —  
4 packets transmitted, 4 received, 0% packet loss, time 3013ms  
rtt min/avg/max/mdev = 0.927/1.935/3.393/0.902 ms  
  
(fotsing@vbox)-[~]  
$
```

```
fotsing@fotsing-ange: ~  
64 octets de 192.168.56.20 : icmp_seq=1 ttl=128 temps=2.19 ms  
64 octets de 192.168.56.20 : icmp_seq=2 ttl=128 temps=1.96 ms  
64 octets de 192.168.56.20 : icmp_seq=3 ttl=128 temps=1.46 ms  
64 octets de 192.168.56.20 : icmp_seq=4 ttl=128 temps=1.25 ms  
64 octets de 192.168.56.20 : icmp_seq=5 ttl=128 temps=0.973 ms  
64 octets de 192.168.56.20 : icmp_seq=6 ttl=128 temps=1.41 ms  
64 octets de 192.168.56.20 : icmp_seq=7 ttl=128 temps=1.26 ms  
64 octets de 192.168.56.20 : icmp_seq=8 ttl=128 temps=1.97 ms  
64 octets de 192.168.56.20 : icmp_seq=9 ttl=128 temps=1.19 ms  
64 octets de 192.168.56.20 : icmp_seq=10 ttl=128 temps=2.67 ms  
64 octets de 192.168.56.20 : icmp_seq=11 ttl=128 temps=1.42 ms  
64 octets de 192.168.56.20 : icmp_seq=12 ttl=128 temps=0.995 ms  
64 octets de 192.168.56.20 : icmp_seq=13 ttl=128 temps=1.22 ms  
^C  
--- statistiques ping 192.168.56.20 ---  
13 paquets transmis, 13 reçus, 0 % paquets perdus, temps 12022 ms  
rtt min/avg/max/mdev = 0.973/1.534/2.666/0.488 ms  
fotsing@fotsing-ange:~$ ping 192.168.56.30  
PING 192.168.56.30 (192.168.56.30) 56(84) bytes of data.  
64 octets de 192.168.56.30 : icmp_seq=1 ttl=64 temps=4.77 ms  
64 octets de 192.168.56.30 : icmp_seq=2 ttl=64 temps=1.31 ms  
64 octets de 192.168.56.30 : icmp_seq=3 ttl=64 temps=2.23 ms  
64 octets de 192.168.56.30 : icmp_seq=4 ttl=64 temps=2.09 ms  
64 octets de 192.168.56.30 : icmp_seq=5 ttl=64 temps=2.87 ms  
^C  
--- statistiques ping 192.168.56.30 ---  
5 paquets transmis, 5 reçus, 0 % paquets perdus, temps 4007 ms  
rtt min/avg/max/mdev = 1.306/2.653/4.768/1.169 ms  
fotsing@fotsing-ange:~$
```

3- Étape 2 : Installation et Mise à jour des bases

Un scanner sans signatures à jour est inefficace.

- Nessus : Activation de la licence Essentials et téléchargement des derniers plugins.
- OpenVAS : Lancement de la mise à jour des flux (feeds) via gym-feed-update.

```

(kali@kali) [~]
$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-04-17 09:52:18 EDT; 1 day 22h ago
 Invocation: c0ad8486f7a64419bddc023d8dcb84a3
 TriggeredBy: ● docker.socket
    Docs: https://docs.docker.com
   Main PID: 542929 (dockerd)
      Tasks: 25
  Memory: 55.8M (peak: 388.2M swap: 14.5M swap peak: 16.4M)
     CPU: 41min 36.525s
    CGroup: /system.slice/docker.service
            └─542929 /usr/sbin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
               └─643383 /usr/sbin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 9392 -container-ip 172.17.0.2 -container-port 9
                  └─643390 /usr/sbin/docker-proxy -proto tcp -host-ip :: -host-port 9392 -container-ip 172.17.0.2 -container-port 9392 -

Apr 18 09:54:09 kali dockerd[542929]: time="2026-04-18T09:54:09.834081592-04:00" level=info msg="Download failed, retrying (4/5): d
Apr 18 09:54:29 kali dockerd[542929]: time="2026-04-18T09:54:29.841297502-04:00" level=info msg="Download failed, retrying (4/5): d
Apr 18 09:54:44 kali dockerd[542929]: time="2026-04-18T09:54:44.841681418-04:00" level=error msg="Download failed after 5 attempts:
Apr 18 09:55:09 kali dockerd[542929]: time="2026-04-18T09:55:09.862924561-04:00" level=error msg="Download failed after 5 attempts:
Apr 18 09:55:09 kali dockerd[542929]: time="2026-04-18T09:55:09.875992250-04:00" level=info msg="Attempting next endpoint for pull
Apr 19 06:12:16 kali dockerd[542929]: time="2026-04-19T06:12:16.278389225-04:00" level=info msg="Download failed, retrying (1/5): u
Apr 19 06:19:37 kali dockerd[542929]: time="2026-04-19T06:19:37.275125764-04:00" level=info msg="Download failed, retrying (2/5): u

```

```

$ sudo docker run -d -p 9392:9392 immauss/openvas --default-url=/web/homepage.html
[sudo] password for kali:
Unable to find image 'immauss/openvas:latest' locally
latest: Pulling from immauss/openvas
6a7e0620566c: Pull complete
aa9d88a882f7: Pull complete
d87b1cb1922a: Pull complete
ab06ad33276c: Pull complete
d7d559d04a17: Pull complete
e2b0d66f291d: Pull complete
1450cfb9c575: Pull complete
ab85775083d2: Pull complete
28bd8a2a75f7: Pull complete
8c44e194e19b: Pull complete
aca74bdf25f: Pull complete
cc495de32044: Pull complete
7ef1fe4a3b34: Pull complete
982770dd7501: Pull complete
52e58d875de5: Pull complete
1f61208972a1: Pull complete
dff1da059c01: Pull complete
175dbd626696: Pull complete
c90007d7cf1b: Pull complete
61792b610b35: Pull complete
9cc2278b0f71: Pull complete
ce20f5083ebb: Pull complete
b9e4a630c52a: Pull complete
7664605793a2: Pull complete
4f4fb700ef54: Pull complete
a4e44cf912c9: Pull complete
30c852f0edc0: Pull complete
23bc60e54bdf: Pull complete
Digest: sha256:b9fc9928d066846cbc8e0a22f882c64a9acf32a8f95c7fbf7d0bff3aff22d792
Status: Downloaded newer image for immauss/openvas:latest

```

```

(kali@kali) [~]
$ sudo systemctl start docker

(kali@kali) [~]
$ sudo systemctl enable docker
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker

```

4- Étape 3 : Configuration du Scan (Discovery)

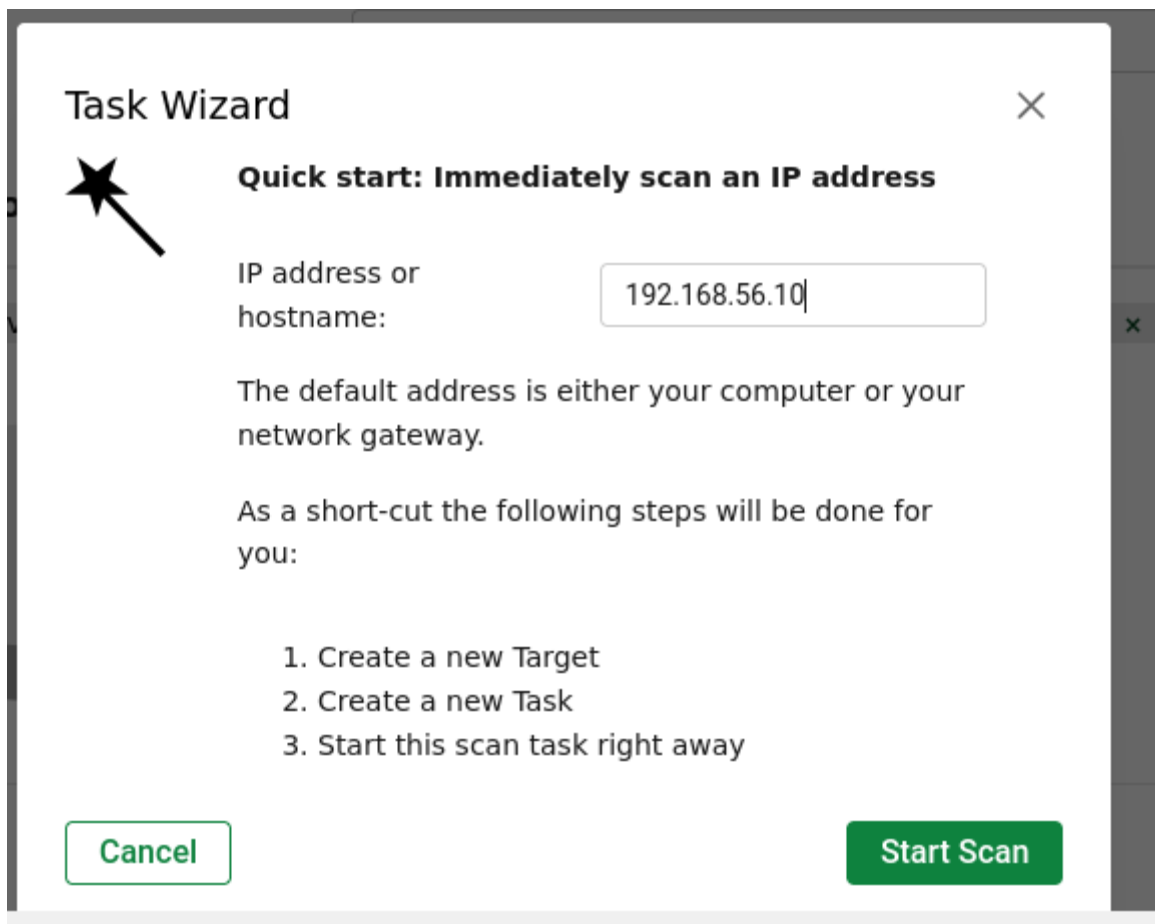
Nous avons défini une politique de scan "Basic Network Scan" pour identifier les services ouverts.

- Paramètres : Détection d'OS, scan des ports TCP/UDP courants (Top 1000).
- Authentification : Configuration des identifiants (SSH pour Linux, SMB pour Windows) pour effectuer un scan en profondeur "à l'intérieur" du système.

5- Étape 4 : Lancement et Analyse des résultats

Une fois les scans terminés, nous analysons les graphiques de sévérité.

- Analyse critique : Recherche des vulnérabilités de score CVSS > 7.0 (Rouge).
- Corrélation : Comparaison entre les résultats de Nessus et OpenVAS pour identifier d'éventuels faux positifs ou des manques.



6- Étape 5 : Focus sur une vulnérabilité majeure (Ex: MS17-010)

Nous isolons une faille spécifique pour comprendre son impact.

- Description : Analyse de la CVE identifiée.
- Solution : Étude de la remédiation suggérée par l'outil (Installation d'un patch KB ou mise à jour de version).

7- Étape 6 : Plan de remédiation (Hardening)

La sécurité ne s'arrête pas au scan. Nous listons les actions prioritaires :

1. Mise à jour immédiate du noyau Linux et des services Windows.
2. Désactivation des protocoles obsolètes détectés (ex: SMBv1, Telnet).
3. Application des règles de pare-feu pour limiter l'exposition.

II- CONCLUSION

Ce TP a démontré que bien que Nessus offre une expérience plus fluide et des rapports plus clairs, OpenVAS reste un outil indispensable pour une surveillance continue sans coût de licence. La maîtrise de ces deux scanners permet à un analyste SOC d'avoir une vision objective de la surface d'attaque. La détection automatisée, couplée à une analyse manuelle, est la seule méthode fiable pour maintenir un niveau de sécurité acceptable dans un parc informatique moderne.